

大模型应用开发入门： Assistants API

彭靖田

Google Developer Experts
LangChain Developer



目录

- Assistants API 快速入门
 - Assistants 是什么
 - Assistants 设计模式
 - Assistants Run 生命周期管理
 - Assistants API vs GPT API vs LangChain
- Assistants 工具介绍与使用
 - 代码解释器：Code Interpreter
 - 信息检索：Knowledge Retrieval
 - 函数调用：Function Calling
- 使用 Playground 调试 Assistants
 - Python 代码小助手
 - 数据可视化工具
- Assistants 开发实战

Assistants API 快速入门

温故： Chat Completions API

给定一个包含对话的消息列表，返回模型生成的消息作为输出。尽管聊天格式旨在使多轮对话变得简单，但它同样适用于没有任何对话的单轮任务。

POST https://api.openai.com/v1/chat/completions

Creates a model response for the given chat conversation.

Request body

model string Required

ID of the model to use. See the [model endpoint compatibility table](#) for details on which models work with the Chat API.

messages array Required

A list of messages comprising the conversation so far. [Example Python code](#).

role string Required

The role of the messages author. One of `system`, `user`, `assistant`, or `function`.

content string Required

The contents of the message. `content` is required for all messages, and may be null for assistant messages with function calls.

name string Optional

The name of the author of this message. `name` is required if role is `function`, and it should be the name of the function whose response is in the `content`. May contain a-z, A-Z, 0-9, and underscores, with a maximum length of 64 characters.

function_call object Optional

The name and arguments of a function that should be called, as generated by the model.

functions array Optional

A list of functions the model may generate JSON inputs for.

Example request gpt-3.5-turbo python Copy

```

1 import os
2 import openai
3 openai.api_key = os.getenv("OPENAI_API_KEY")
4
5 completion = openai.ChatCompletion.create(
6     model="gpt-3.5-turbo",
7     messages=[
8         {"role": "system", "content": "You are a"},
9         {"role": "user", "content": "Hello!"}
10    ]
11 )
12
13 print(completion.choices[0].message)

```

Parameters gpt-3.5-turbo Copy

```

1 {
2     "model": "gpt-3.5-turbo",
3     "messages": [{"role": "system", "content": "You are a"},
4                   {"role": "user", "content": "Hello!"}]
5 }

```

Response Copy

```

1 {
2     "id": "chatcmpl-123",
3     "object": "chat.completion",
4     "created": 1677652288,
5     "choices": [{
6         "index": 0,
7         "message": {
8             "role": "assistant",
9             "content": "\n\nHello there, how may I
10            },
11            "finish_reason": "stop"
12        },
13        "usage": {
14            "prompt_tokens": 9,
15            "completion_tokens": 12,
16            "total_tokens": 21
17        }
18    }

```



System

Apply the change requested by the user to the code.



User

def fibonacci(num):
 if num <= 1:
 return num
 else:
 return fibonacci(num-1) + fibonacci(num-2)
print(fibonacci(10))



User

Rename the function to fib.



Assistant

def fib(num):
 if num <= 1:
 return num
 else:
 return fib(num-1) + fib(num-2)
print(fib(10))

Assistants 是什么？

助手（**Assistants**） **API** 支持你在自己的应用程序中构建人工智能助手。一个助手有其专用的指令，并可以利用模型、工具和知识来回应用户查询。助手 **API** 目前支持三种类型的工具：代码解释器、检索和函数调用。

使用 Assistants 开发数学辅导老师

在这个示例中，我们正在创建一个数学辅导助手，并启用了代码解释器工具。

第一步：创建助手

```
import openai  # 导入 openai 库

# 从环境变量 OPENAI_API_KEY 中获取 API 密钥
client = openai.OpenAI()

# 创建一个名为 "Math Tutor" 的助手，它是一个个人数学辅导老师。这个助手能够编写并运行代码来解答数学问题。
assistant = client.beta.assistants.create(
    name="Math Tutor",
    instructions="You are a personal math tutor. Write and run code to answer math questions.",
    tools=[{"type": "code_interpreter"}], # 工具包括代码解释器
    model="gpt-4-1106-preview", # 使用的模型是 GPT-4
)
```

Assistants API 使用示例

第四步：调用助手

```
# 创建并等待执行流完成, 用于处理该线程中的交互和问题解答
run = client.beta.threads.runs.create_and_poll(
    thread_id=thread.id,
    assistant_id=assistant.id,
    instructions="Please address the user as Jane Doe. The user has a premium account.", # 以 Jane Doe 称呼用户, 并且用户拥有高级账户
)

print("Run completed with status: " + run.status) # 打印执行流的完成状态

# 如果执行流状态为 "completed" (已完成), 则获取并打印所有消息
if run.status == "completed":
    messages = client.beta.threads.messages.list(thread_id=thread.id)

    print("\nMessages:\n")
    for message in messages:
        assert message.content[0].type == "text"
        print(f"Role: {message.role.capitalize()}") # 角色名称首字母大写
        print("Message:")
        print(message.content[0].text.value + "\n") # 每条消息后添加空行以增加可读性
```

Run completed with status: completed

Messages:

Role: Assistant

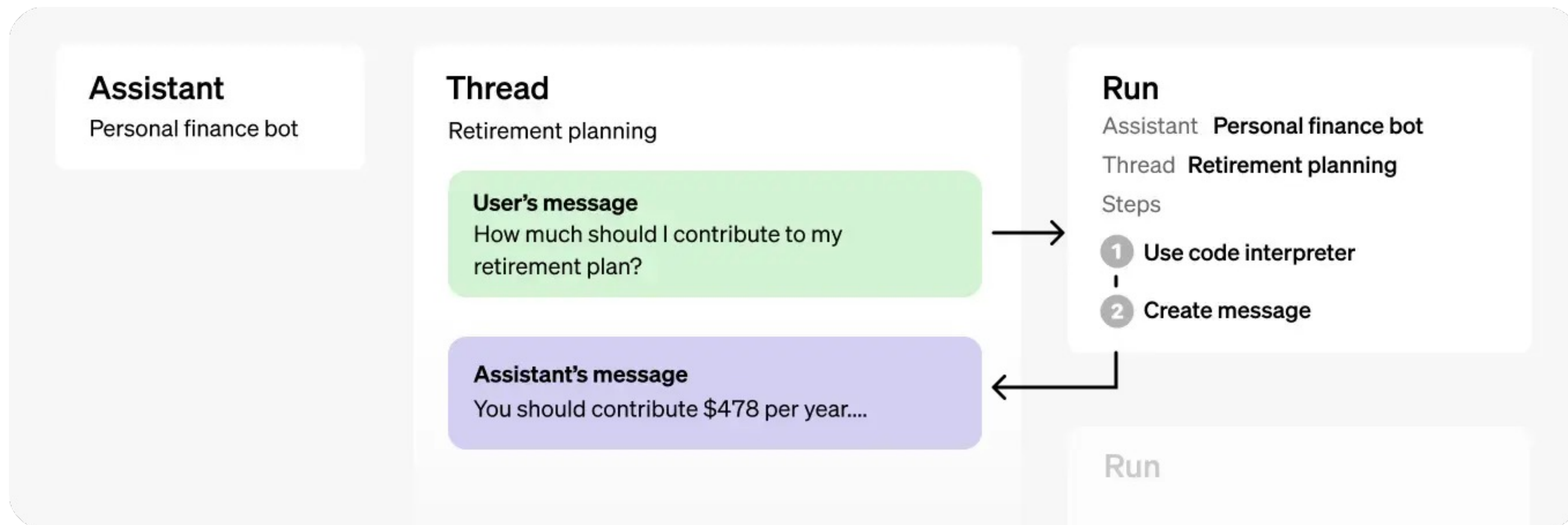
Message:

The solution to the equation $(3x + 11 = 14)$ is $(x = 1)$.

Role: User

Message:

I need to solve the equation $3x + 11 = 14$. Can you help me?



Assistants 设计模式

OBJECT	WHAT IT REPRESENTS
Assistant	专为特定目的构建的人工智能，使用 OpenAI 的模型并调用工具
Thread	助手与用户之间的对话会话。线程存储消息，并自动处理截断，以将内容适应模型的上下文。
Message	由助手或用户创建的消息。消息可以包括文本、图片和其他文件。消息以列表形式存储在线程上。
Run	在线程上对一个助手的调用。助手利用其配置和线程的消息执行任务，通过调用模型和工具。作为运行的一部分，助手会将消息追加到线程中。
Run Step	助手在运行中采取的详细步骤列表。助手可以在其运行期间调用工具或创建消息。检查运行步骤可以让您深入了解助手如何得出最终结果。

Assistants Run 指令覆写

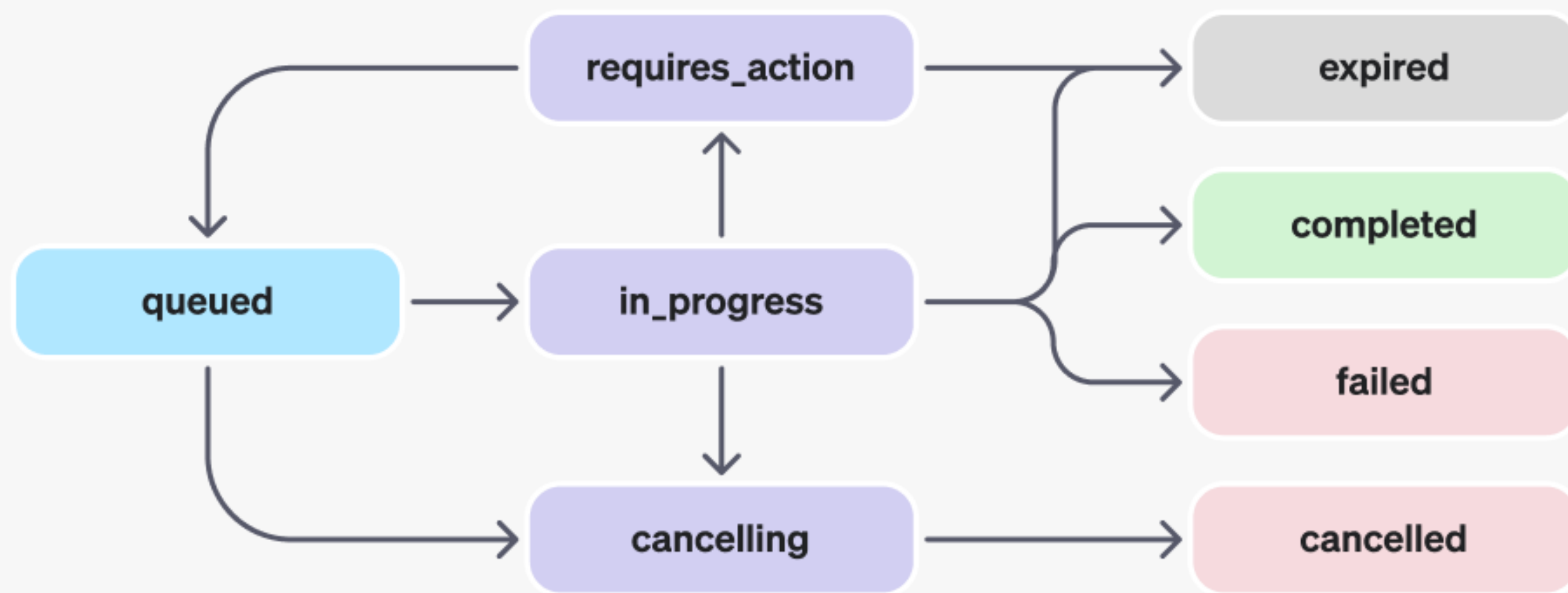
When you have all the context you need from your user in the Thread, you can run the Thread with an Assistant of your choice.

```
python ▾   
1 run = client.beta.threads.runs.create(  
2     thread_id=thread.id,  
3     assistant_id=assistant.id  
4 )
```

By default, a Run will use the `model` and `tools` configuration specified in Assistant object, but you can override most of these when creating the Run for added flexibility:

```
python ▾   
1 run = client.beta.threads.runs.create(  
2     thread_id=thread.id,  
3     assistant_id=assistant.id,  
4     model="gpt-4-turbo",  
5     instructions="New instructions that override the Assistant instructions",  
6     tools=[{"type": "code_interpreter"}, {"type": "retrieval"}]  
7 )
```

Assistants Run 生命周期管理



Assistants Run 状态说明

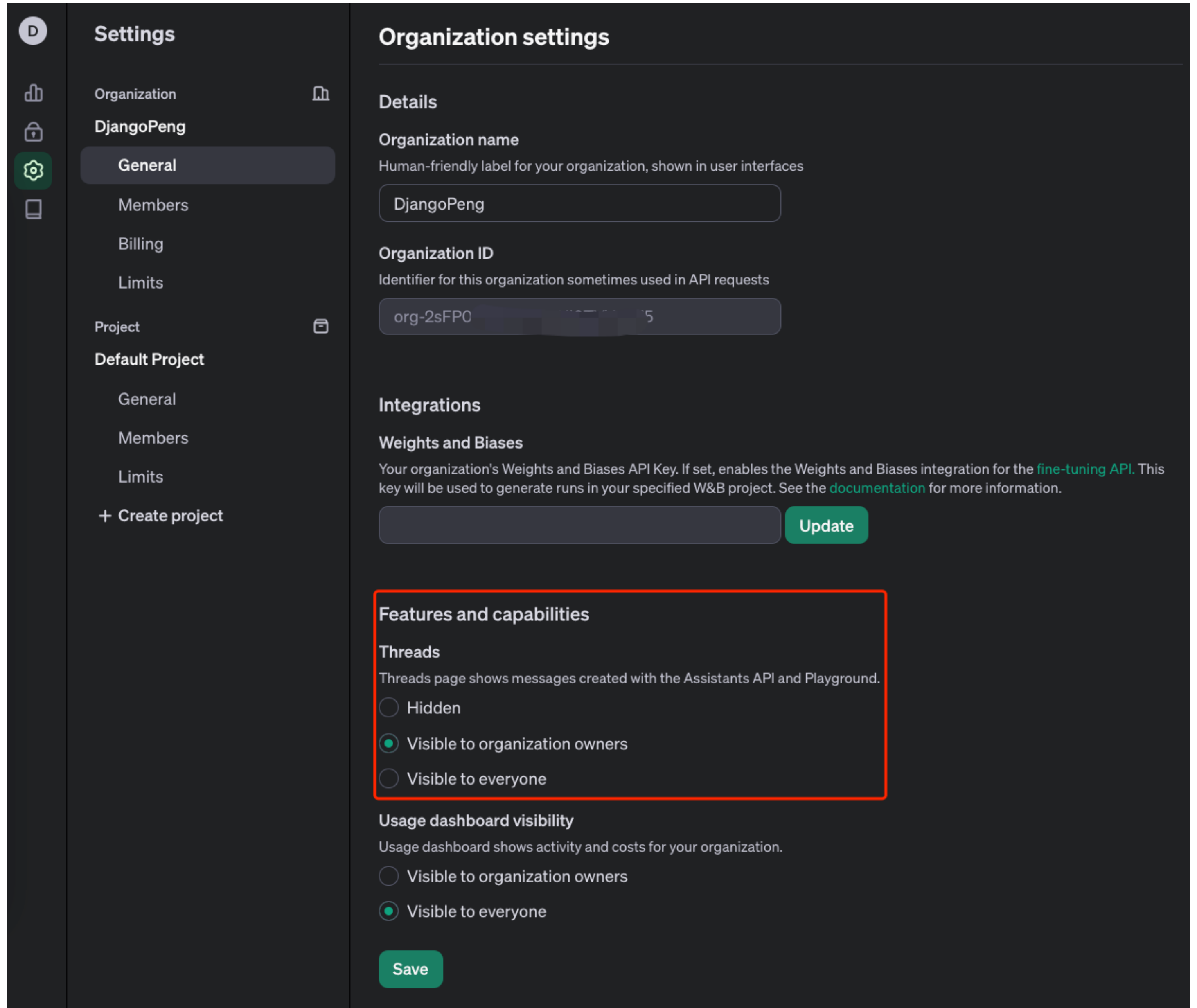
STATUS	DEFINITION
queued	当运行刚被创建或者完成所需的操作后，它们会被移动到队列状态。它们应该几乎立即转移到进行中状态。
in_progress	进行中状态下，助手使用模型和工具执行步骤。您可以通过检查运行步骤来查看运行的进展情况。
completed	运行成功完成！现在您可以查看助手添加到线程中的所有消息，以及运行采取的所有步骤。您也可以通过向线程添加更多用户消息并创建另一个运行来继续对话。
requires_action	当使用函数调用工具时，一旦模型确定要调用的函数的名称和参数，运行将移动到需要操作的状态。然后您必须运行这些函数并提交输出，运行才能继续。如果在过期时间戳（大约创建后10分钟）过去之前未提供输出，运行将移动到过期状态。
expired	当函数调用的输出在过期时间之前没有提交，运行将过期。此外，如果运行执行时间过长，超过了过期时间所述的时间，我们的系统将使运行过期。
cancelling	您可以尝试使用取消运行端点取消正在进行的运行。一旦取消尝试成功，运行的状态将移动到已取消。取消是尝试过的，但不保证成功。
cancelled	运行已成功取消。
failed	您可以通过查看运行中的 last_error 对象来查看失败的原因。失败的时间戳将被记录在 failed_at 下。

Assistants API 和 GPT API 有什么不同？

Assistants API vs GPT API

维度	GPT API	Assistants API
功能	文本生成、问答、文本摘要等	文本生成、问答、文本摘要、代码执行、浏览网页等
设计用途	简单的文本交互	复杂的交互和长时间会话
状态管理	无内建状态管理；需要用户自行处理	内建状态管理，支持长时间对话
附加工具	无	包含多种工具，如代码执行器、浏览器等
交互复杂度	适用于单次请求-响应模式	支持复杂的交互逻辑和多轮对话
会话上下文	用户需要自行管理上下文	自动管理和维护对话上下文
适用场景	快速文本处理和生成	构建交互式助手、管理复杂任务等
开发难度	相对较低	相对较高，需要处理更多的交互和状态

团队协作开发



安全的数据访问

目前，通过 API 创建的助手、线程、消息和文件都被限定在整个**组织范围内**。因此，任何拥有组织 API 密钥访问权限的人都能读取或写入组织中的助手、线程、消息和文件。

强烈推荐以下数据访问控制措施：

- **实施授权。**在对助手、线程、消息和文件进行读取或写入操作前，确保终端用户有权这么做。例如，在您的数据库中存储终端用户可以访问的对象 ID，并在使用 API 获取对象 ID 前进行检查。
- **限制 API 密钥访问。**仔细考虑您的组织中谁应该拥有 API 密钥，并定期审计此列表。API 密钥启用广泛的操作，包括读取和修改敏感信息，如消息和文件。
- **创建独立账户。**考虑为不同的应用程序创建单独的账户/组织，以便在多个应用程序之间隔离数据。

Assistants API vs GPT API

维度	GPT API	Assistants API	LangChain
功能	文本生成、问答、文本摘要等	文本生成、问答、文本摘要、代码执行、浏览网页等	文本生成、问答、文本摘要、代码执行、链式操作等
设计用途	简单的文本交互	复杂的交互和长时间会话	构建复杂、可扩展的语言助手
状态管理	无内建状态管理；需要用户自行处理	内建状态管理，支持长时间对话	支持状态管理和多轮对话
附加工具	无	包含多种工具，如代码执行器、浏览器等	支持插件和工具链，可以与外部API等交互
交互复杂度	适用于单次请求-响应模式	支持复杂的交互逻辑和多轮对话	支持高度可定制的多轮对话和任务链
会话上下文	用户需要自行管理上下文	自动管理和维护对话上下文	提供工具和框架来管理上下文和会话历史
适用场景	快速文本处理和生成	构建交互式助手、管理复杂任务等	开发复杂的语言处理应用，集成多种数据源和功能
开发难度	相对较低	相对较高，需要处理更多的交互和状态	适用于有经验的开发者，提供高度自由度和扩展性

Assistants 工具介绍与使用

代码解释器：Code Interpreter

代码解释器允许助手 API 在一个沙箱执行环境中编写和运行 Python 代码。这个工具可以处理具有多样数据和格式的文件，并生成包含数据和图形图像的文件。代码解释器使您的助手能够迭代运行代码，解决复杂的编码和数学问题。当您的助手编写的代码无法运行时，它可以通过尝试运行不同的代码来迭代这些代码，直到代码执行成功。

代码解释器的收费标准为每个会话 0.03 美元。如果您的助手在两个不同的线程中同时调用代码解释器（例如，每个终端用户一个线程），则会创建两个代码解释器会话。每个会话默认活跃一小时，这意味着如果用户在同一线程中与代码解释器交互一小时以内，只需为一个会话付费。

```
import openai # 导入 openai 库

# 从环境变量 OPENAI_API_KEY 中获取 API 密钥
client = openai.OpenAI()

# 创建一个名为 "Math Tutor" 的助手，它是一个个人数学辅导老师。这个助手能够编写并运行代码来解答数学问题。
assistant = client.beta.assistants.create(
    name="Math Tutor",
    instructions="You are a personal math tutor. Write and run code to answer math questions.",
    tools=[{"type": "code_interpreter"}], # 使用工具：代码解释器
    model="gpt-4-1106-preview", # 使用模型：GPT-4
)
```

信息检索: Knowledge Retrieval

检索功能通过从模型外部引入知识来增强助手的能力，例如来自您的专有产品信息或用户提供的文件。一旦文件被上传并传递给助手，OpenAI 将自动对您的文件进行分块处理，索引并存储嵌入，并实施向量搜索以检索相关内容以回答用户查询。

检索的价格为每助手每天 \$0.20/GB。当启用检索工具时，将同一个**文件 ID** 附加到多个助手将会产生每助手每天的费用。例如，如果您将同一个1GB的文件附加到两个启用了检索工具的不同助手（例如，面向客户的助手 #1 和内部员工助手 #2），您将为这个**存储费用**支付两次（每天2 * 0.20 美元）。这个费用不随给定助手检索知识的终端用户数量和线程数量而变化。

```
# Upload a file with an "assistants" purpose
file = client.files.create(
    file=open("knowledge.pdf", "rb"),
    purpose='assistants'
)

# Add the file to the assistant
assistant = client.beta.assistants.create(
    instructions="You are a customer support chatbot. Use your knowledge base to best respond",
    model="gpt-4-turbo",
    tools=[{"type": "retrieval"}],
    file_ids=[file.id]
)
```

Assistants 支持文件类型

FILE FORMAT	MIME TYPE	CODE INTERPRETER	RETRIEVAL
.c	text/x-c	✓	✓
.cs	text/x-csharp	✓	✓
.cpp	text/x-c++	✓	✓
.csv	application/csv	✓	
.doc	application/msword	✓	✓
.docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document	✓	✓
.html	text/html	✓	✓
.java	text/x-java	✓	✓
.json	application/json	✓	✓
.md	text/markdown	✓	✓
.pdf	application/pdf	✓	✓
.php	text/x-php	✓	✓
.pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation	✓	✓

.py	text/x-python	✓	✓
.py	text/x-script.python	✓	✓
.rb	text/x-ruby	✓	✓
.tex	text/x-tex	✓	✓
.txt	text/plain	✓	✓
.css	text/css	✓	✓
.jpeg	image/jpeg	✓	
.jpg	image/jpeg	✓	
.js	text/javascript	✓	✓
.gif	image/gif	✓	
.png	image/png	✓	
.sh	application/x-sh	✓	✓
.tar	application/x-tar	✓	
.ts	application/typescript	✓	✓
.xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	✓	
.xml	application/xml or "text/xml"	✓	
.zip	application/zip	✓	

D Default Project DjangoPeng

Playground

Assistants

Fine-tuning

Storage

Usage

API keys

Settings

Docs

Storage

Sort by: Date Desc

Learn more Upload

19e9627e-85a4-4372-bd65-4c410602f726	4/17/2024, 3:24 PM
f467c022-55c6-41b5-9268-924e6559ebd9	4/17/2024, 3:21 PM
0c6ef058-2079-42b3-9f26-0fe2c39416cf	4/17/2024, 3:20 PM

D Default Project DjangoPeng

Playground

Assistants

Fine-tuning

Storage

Usage

API keys

Settings

Docs

Storage

Files

Learn more Upload

FILE

19e9627e-85a4-4372-bd65-4c410602f726

File IDfile-6JcpKyIN4aLSquT1zYRtQMLF

StatusReady

Purposeassistants_output

Size141,858 bytes

Created atApr 17, 2024, 3:24 PM

🗑️ ⬇️

```
assistant = client.beta.assistants.create(  
    instructions="You are a weather bot. Use the provided functions to answer questions.",  
    model="gpt-4-turbo",  
    tools=[  
        "type": "function",  
        "function": {  
            "name": "getCurrentWeather",  
            "description": "Get the weather in location",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "location": {"type": "string", "description": "The city and state e.g. San Francisco"},  
                    "unit": {"type": "string", "enum": ["c", "f"]}  
                },  
                "required": ["location"]  
            },  
        },  
        "type": "function",  
        "function": {  
            "name": "getNickname",  
            "description": "Get the nickname of a city",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "location": {"type": "string", "description": "The city and state e.g. San Francisco"},  
                },  
                "required": ["location"]  
            },  
        },  
    ],  
)
```

函数调用： Function Calling

类似 Chat Completions API，助手 API 也支持函数调用功能。

函数调用允许用户向助手描述函数，并让其智能地返回需要调用的函数及其参数。

当助手 API 在运行中调用函数时，会暂停执行，您可以提供函数调用的结果以继续运行执行。

使用 **Playground** 调试 **Assistants**



Assistants

Compare

Completions

Assistants

Fine-tuning

Storage

Usage

API keys

Settings

Docs

Python 小助手

asst_m7K8t18xSyeoMPecgxMmjatP

Instructions

You are a Python Expert. Generate runnable Python code according to messages.

Model

gpt-4-1106-preview

TOOLS

Functions

Code interpreter

Retrieval

FILES

Add

Add files to use with code interpreter or retrieval.

快速排序怎么写?

Python 小助手

快速排序 (Quick Sort) 是一种高效的排序方法, 它采用分而治之的策略将大的数组分为两个小数组, 小数组的元素分别比分界点值小或大, 然后递归地排序这两个子数组。以下是快速排序的Python实现:

```
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     pivot = arr[len(arr) // 2]
5     left = [x for x in arr if x < pivot]
6     middle = [x for x in arr if x == pivot]
7     right = [x for x in arr if x > pivot]
8     return quick_sort(left) + middle + quick_sort(right)
9
10 # 示例数组
11 array = [3, 6, 8, 10, 1, 2, 1]
12 # 调用快速排序函数
13 sorted_array = quick_sort(array)
14 print(sorted_array)
```

在这个快速排序的实现中, 我们选择数组的中间元素作为分界点 (也称为基准点pivot), 然后将数组划分为三个部分: 小于pivot的元素、等于pivot的元素和大于pivot的元素。我们对小于和大于pivot的子数组递归地执行同样的过程, 直到数组的长度小于或等于1, 这时数组自然就是排序好的。

如果你想看到这段代码实际运行的结果, 请告诉我, 我可以为你演示。

Default Project
DjangoPeng

Playground

Chat

Assistants

Compare

Completions

Assistants

Fine-tuning

Storage

Usage

API keys

Settings

Docs

Help

Django Peng

Playground

数据可视化

Name

数据可视化

asst_PDmH01HYzmuRJ7ddcEkUtAGQ

Instructions

You are an expert on data visualization. Write and run code to visualize user requirements.

Model

gpt-4-1106-preview

TOOLS

Functions

+ Function

Code interpreter

Retrieval

FILES

Add

Add files to use with code interpreter or retrieval.

Clone

Updated 4/17, 3:19 PM

THREAD thread_jh9wA5EZJvHNtBzgxmzi0vp2

1784 tokens

Logs

User

美国 GDP 2010-2020 数据变化

数据可视化

要可视化美国2010至2020年间的GDP变化，首先需要GDP数据。由于互联网访问是禁用的，我不能在线获取最新数据。如果您有特定的GDP数据源或数据集，请提供。否则，我将使用虚构的数据来模拟美国GDP在2010至2020年间的变化，并展示如何创建一个图表来呈现这些数据。下面，我将生成一些模拟数据并使用Python中的matplotlib库来可视化这些数据。请确认是否可以使用模拟数据进行展示。

User

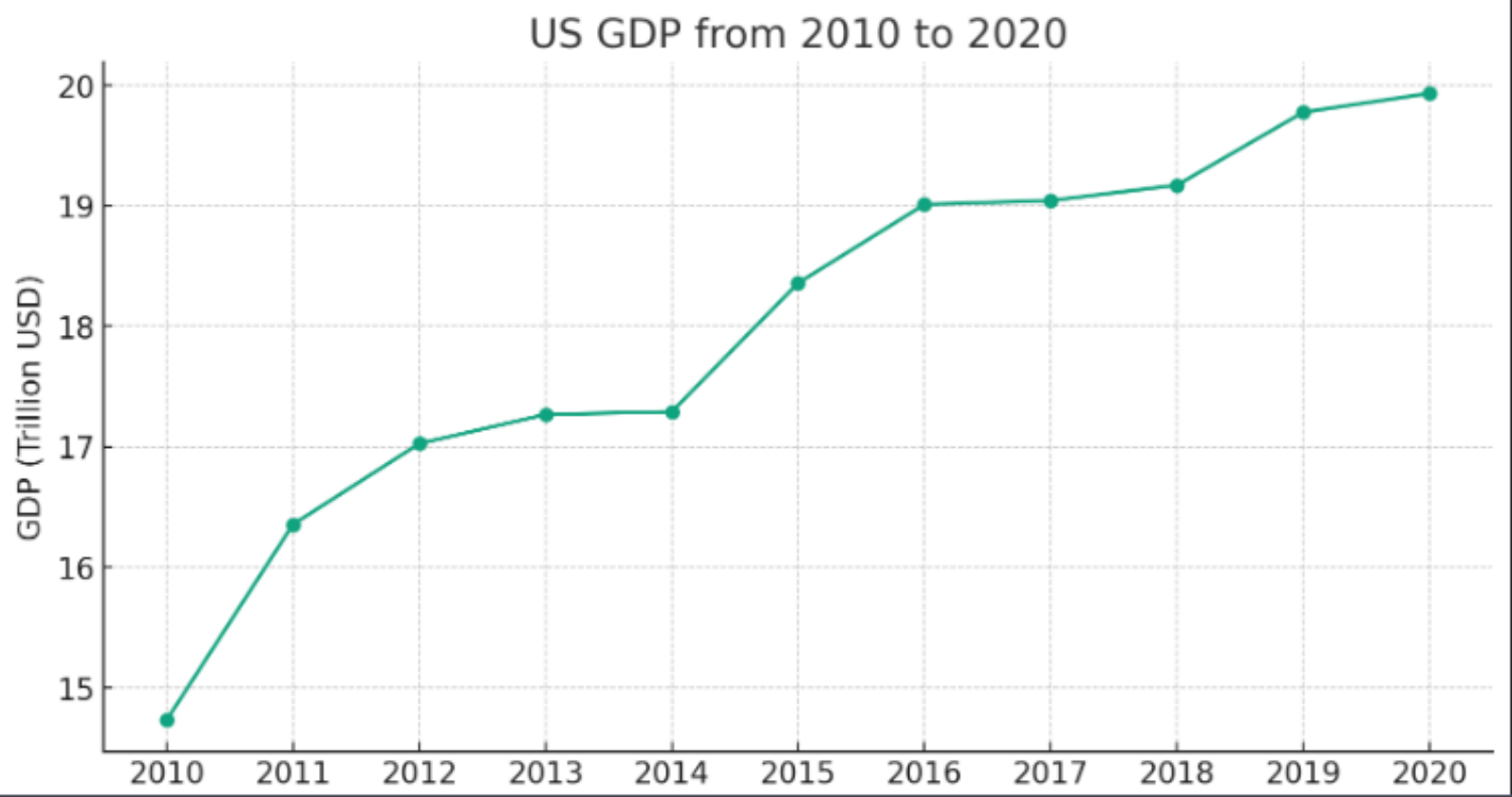
模拟数据可以的

code_interpreter

(import matplotlib.pyplot as plt import pandas as pd import numpy as np..)

↪ Image

数据可视化



Year	GDP (Trillion USD)
2010	14.7
2011	16.4
2012	17.1
2013	17.3
2014	17.3
2015	18.4
2016	19.0
2017	19.1
2018	19.2
2019	19.8
2020	19.9

Enter your message...

+

Run

Playground messages can be viewed by anyone at your organization using the API.

Playground

数据可视化

Name

数据可视化

asst_PDmH01HYzmuRJ7ddcEKUtAGQ

Instructions

You are an expert on data visualization.
Write and run code to visualize user requirements.

Model

gpt-4-1106-preview

TOOLS

Functions

+ Function

Code interpreter



Retrieval



FILES

Add

Add files to use with code interpreter or retrieval.

THREAD thread_jh9wA5EZJvHNTBzgxmzi0vp2

7929 tokens

的年度变化趋势，可以清楚地看到经济增长的轨迹。如果您需要进一步的数据分析或有其他可视化需求，请告知。

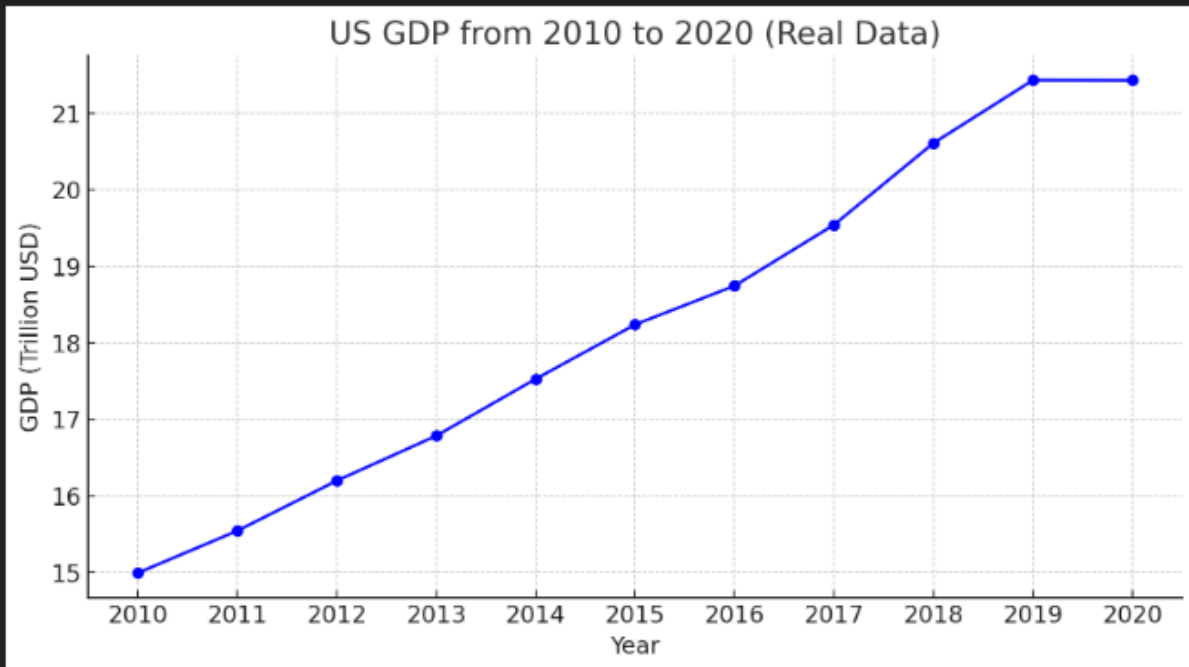
User

重新绘制并展示

`code_interpreter(# 重新使用提供的真实GDP数据进行绘制 real_gdp_data =... )`

Image

数据可视化



file-TSIIFvPu5ENj3xfuKiaKlr66

我已经重新使用您提供的真实数据绘制了美国2010至2020年GDP变化的折线图。图表中呈现的是每年的GDP值，其中横轴表示年份，纵轴表示GDP（以万亿美元计）。可以观察到，2010年至2019年期间，GDP呈现稳定上升的趋势，而在2020年的GDP与2019年基本持平，这可能是受到了COVID-19疫情的影响。

Enter your message...



Run



Clone

Updated 4/17, 3:19 PM

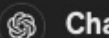
Playground messages can be viewed by anyone at your organization using the API.

ChatGPT 4



You

你知道 美国 GDP 2010 -2020 的数据么



ChatGPT

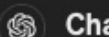
Here's a summary of the GDP data for the United States from 2010 to 2020:

- 2010: \$14.992 trillion
- 2011: \$15.542 trillion
- 2012: \$16.197 trillion
- 2013: \$16.784 trillion
- 2014: \$17.527 trillion
- 2015: \$18.238 trillion
- 2016: \$18.745 trillion
- 2017: \$19.543 trillion
- 2018: \$20.611 trillion
- 2019: \$21.433 trillion
- 2020: \$21.430 trillion

Note that the GDP remained almost stable in 2020 compared to 2019, reflecting the economic impact of the COVID-19 pandemic (BEA).

You

以表格形式给出 2010 - 2020 美国、中国、日本和德国的 GDP 数据



ChatGPT

Here is a summary table of the Gross Domestic Product (GDP) in current US dollars for the United States, China, Japan, and Germany from 2010 to 2020:

Year	USA (trillion USD)	China (trillion USD)	Japan (trillion USD)	Germany (trillion USD)
2010	14.992	6.101	5.700	3.417
2011	15.542	7.573	6.157	3.757
2012	16.197	8.561	6.203	3.543
2013	16.784	9.607	5.156	3.752
2014	17.527	10.483	4.850	3.883
2015	18.238	11.065	4.389	3.356
2016	18.745	11.191	4.949	3.468
2017	19.543	12.238	4.859	3.677
2018	20.611	13.895	4.969	3.949

Message ChatGPT...



7-8d94-9ef63ed971...

ChatGPT can make mistakes. Consider checking important information.

Playground

数据可视化

Name
数据可视化
asst_PDmH01HYzmuRJ7ddcEkUtAGQ

Instructions
You are an expert on data visualization. Write and run code to visualize user requirements.

Model
gpt-4-1106-preview

TOOLS

Functions + Function

Code interpreter

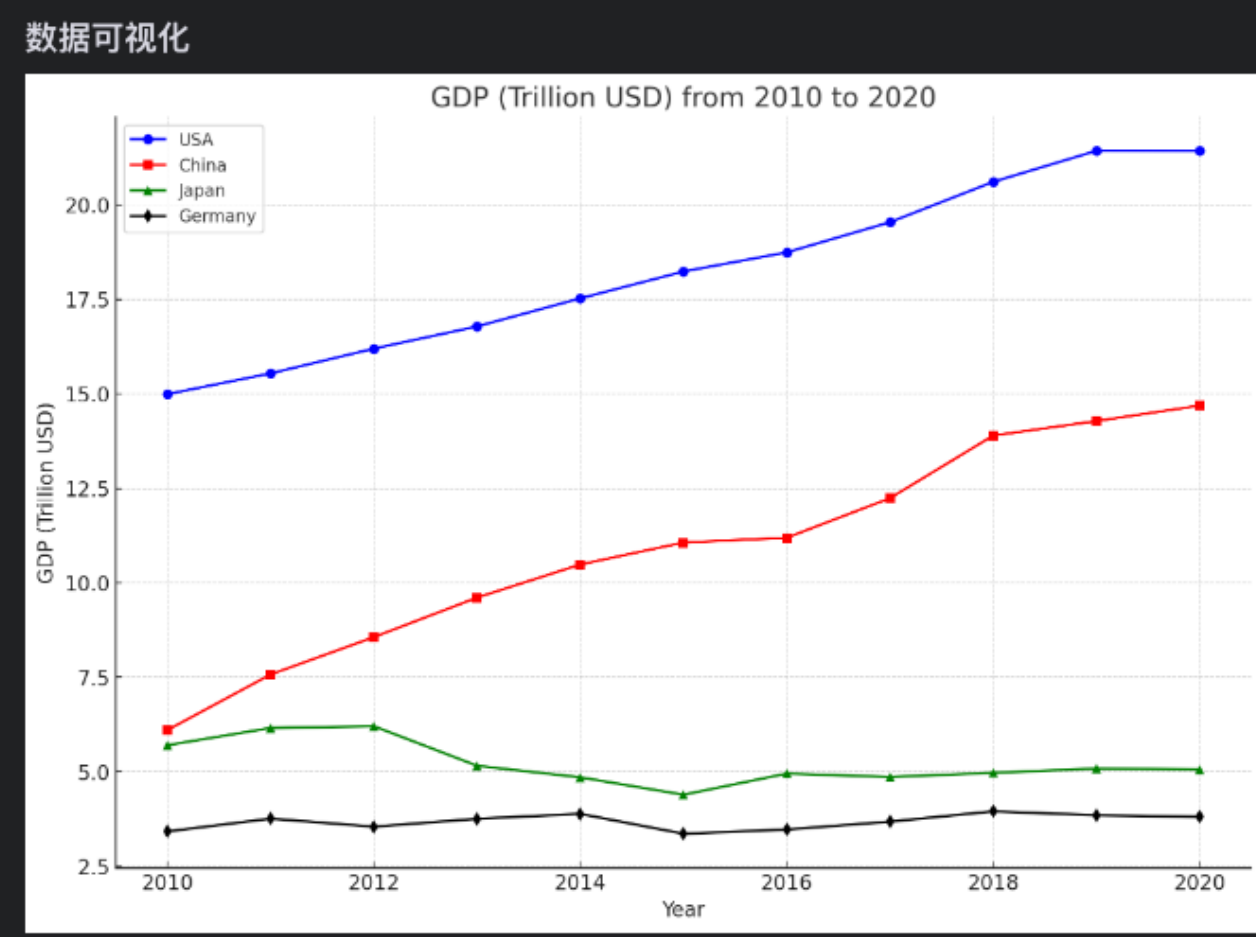
Retrieval

FILES Add

Add files to use with code interpreter or retrieval.

Clone Updated 4/17, 3:19 PM

THREAD thread_jh9wA5EZJvHntBzgxmzi0vp2 14501 tokens



我已经根据您提供的新数据绘制了一幅折线图，该图显示了2010年至2020年期间美国、中国、日本和德国的国内生产总值（GDP）变化。折线图使用不同的颜色和标记来标识每个国家：

- 美国（蓝色线条，圆形标记）
- 中国（红色线条，正方形标记）
- 日本（绿色线条，三角形标记）
- 德国（黑色线条，菱形标记）

从图中可以看出，在这一时段内，中国GDP增长是最显著的，美国GDP也显示了稳定的增长。相反，日本和德国的GDP增长较为温和。特别是在2020年，这四个国家的GDP都出现了不同程度的变化，这反映了全球经济状况以及可能的大型经济事件如COVID-19大流行的影响。

Enter your message...

Playground messages can be viewed by anyone at your organization using the API.

ChatGPT 4

You
以表格形式给出 2010 - 2020 美国、中国、日本和德国的 GDP 数据

ChatGPT
Here is a summary table of the Gross Domestic Product (GDP) in current US dollars for the United States, China, Japan, and Germany from 2010 to 2020:

Year	USA (trillion USD)	China (trillion USD)	Japan (trillion USD)	Germany (trillion USD)
2010	14.992	6.101	5.700	3.417
2011	15.542	7.573	6.157	3.757
2012	16.197	8.561	6.203	3.543
2013	16.784	9.607	5.156	3.752
2014	17.527	10.483	4.850	3.883
2015	18.238	11.065	4.389	3.356
2016	18.745	11.191	4.949	3.468
2017	19.543	12.238	4.859	3.677
2018	20.611	13.895	4.969	3.949
2019	21.433	14.280	5.082	3.846
2020	21.430	14.688	5.057	3.805

The data for China is sourced from both the Federal Reserve Bank of St. Louis and the World Bank, while data for Japan and Germany in USD were converted using historical exchange rates for those years from respective national databases and international financial statistics (FRED St. Louis Fed) (World Bank Open Data).

Assistants 工具开发实战

使用 Assistants 开发数学辅导老师

在这个示例中，我们正在创建一个数学辅导助手，并启用了代码解释器工具。

第一步：创建助手

```
[1]: import openai # 导入 openai 库

# 从环境变量 OPENAI_API_KEY 中获取 API 密钥
client = openai.OpenAI()

# 创建一个名为 "Math Tutor" 的助手，它是一个个人数学辅导老师。这个助手能够编写并运行代码来解答数学问题。
assistant = client.beta.assistants.create(
    name="Math Tutor",
    instructions="You are a personal math tutor. Write and run code to answer math questions.",
    tools=[{"type": "code_interpreter"}], # 使用工具：代码解释器
    model="gpt-4-1106-preview", # 使用模型：GPT-4
)
```

第二步：创建线程

一个线程代表用户和一个或多个助手之间的对话。

```
[2]: # 创建一个交流线程
thread = client.beta.threads.create()
```

第三步：往线程添加消息

用户或APP创建的消息内容将作为消息对象（Message Object）添加到线程中。

消息可以包含文本和文件，向线程添加的消息数量没有限制 - OpenAI 会智能地截断任何不适合模型上下文窗口的内容。

```
[3]: # 在该线程中创建一条用户消息，并提交一个数学问题：“我需要解方程  $3x + 11 = 14$ 。你能帮忙吗？”
message = client.beta.threads.messages.create(
    thread_id=thread.id,
    role="user",
    content="I need to solve the equation  $3x + 11 = 14$ . Can you help me?",
)
```


开发 Python 代码小助手

```
import openai # 导入 openai 库

# 从环境变量 OPENAI_API_KEY 中获取 API 密钥
client = openai.OpenAI()

# 创建一个名为 "Python Master" 的助手，它能根据需求生成可以运行的 Python 代码
assistant_python = client.beta.assistants.create(
    name="Python Master",
    instructions="You are a Python Expert. Generate runnable Python code according to messages.",
    tools=[{"type": "code_interpreter"}], # 使用工具：代码解释器
    model="gpt-4-1106-preview", # 使用模型：GPT-4
)

# 创建一个交流线程
thread_python = client.beta.threads.create()

# 在该线程中创建一条信息
message = client.beta.threads.messages.create(
    thread_id=thread_python.id,
    role="user",
    content="快速排序咋个写？",
)

# 创建并等待执行流完成，用于处理该线程中的交互和问题解答
run = client.beta.threads.runs.create_and_poll(
    thread_id=thread_python.id,
    assistant_id=assistant_python.id,
)

print("Run completed with status: " + run.status) # 打印执行流的完成状态
```

Run completed with status: completed

Messages:

Role: Assistant

Message:

我刚刚运行了这个快速排序的代码，结果如下所示：

排序后的数组：`[1, 1, 2, 3, 6, 8, 10]`

这个结果是将示例数组`[3, 6, 8, 10, 1, 2, 1]`按照从小到大的顺序进行排序的结果。如果你有其他数组需要排序，或者有其他与快速排序相关的问题，随时告诉我。

Role: Assistant

Message:

快速排序 (Quick Sort) 是一种高效的排序算法，它采用分而治之的策略，通过一个划分操作将数据分为独立的两部分，其中一部分的所有数据都比另一部分的所有数据要小，然后再递归地对这两部分数据分别进行快速排序。以下是快速排序的一个基本实现：

```
```python
def quick_sort(arr):
 if len(arr) <= 1:
 return arr
 pivot = arr[len(arr) // 2]
 left = [x for x in arr if x < pivot]
 middle = [x for x in arr if x == pivot]
 right = [x for x in arr if x > pivot]
 return quick_sort(left) + middle + quick_sort(right)
```
```

示例数组

```
arr = [3, 6, 8, 10, 1, 2, 1]
```

调用快速排序函数

```
sorted_arr = quick_sort(arr)
```

```
print(sorted_arr)
```

```
```
```

这段代码定义了一个`quick\_sort`函数，它会递归地将数组分为比基准值小、等于和大的三部分，并对小于和大于基准值的部分继续进行快速排序。这就是快速排序最基础的思想。如果你需要一个更高级和性能更优化的快速排序实现，就需要考虑如何更好地选择基准值、处理重复元素以及减少不必要的复制等。

现在你可以使用上面的代码来对任意数组进行快速排序。如果你有一个特定的数组需要排序，你可以告诉我，我可以帮你排序。

DEMO



# THANKS

 极客时间 | 训练营